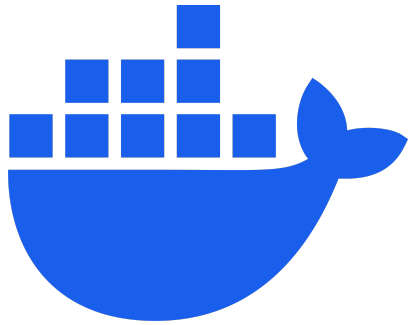




Workshop: Hands-on with Podman

Container Orchestration Workshop - Session 2

November 17, 2025 | 13:00 - 16:00

Speaker: Warut Chomvorathayee
Senior Cloud Engineer & Technical Consultant

Agenda ช่วงบ่าย

1.  ติดตั้ง Podman
2.  คำสั่งพื้นฐานและ Podman Machine
3.  Build Image with Dockerfile
4.  Workshop 1: Hello Node.js
5.  Workshop 2: API with NestJS
6.  Workshop 3: Database + Compose
7.  Docker Networks
8.  Q&A และสรุป

ส่วนที่ 1: ติดตั้ง Podman

Podman คืออะไร?

Podman = Pod Manager (ทางเลือกแทน Docker)

ข้อดีของ Podman

-  **Daemonless** - ไม่ต้องมี daemon รันอยู่ตลอดเวลา
-  **Rootless** - รันได้โดยไม่ต้องใช้ root privileges
-  **Docker Compatible** - คำสั่งเหมือนกับ Docker
-  **Pod Support** - รองรับ Kubernetes pod

คำสั่งเหมือนกับ Docker

```
docker run nginx    →    podman run nginx
docker ps           →    podman ps
docker build         →    podman build
```

การติดตั้ง Podman - Windows

วิธีที่ 1: ดาวน์โหลด Podman Desktop

```
# เข้าไปที่  
https://podman-desktop.io/downloads
```

วิธีที่ 2: ใช้ Chocolatey

```
choco install podman-desktop
```

วิธีที่ 3: ใช้ winget

```
winget install -e --id RedHat.Podman-Desktop
```

การติดตั้ง Podman - macOS

ใช้ Homebrew

```
# ติดตั้ง Podman CLI  
brew install podman  
  
# ติดตั้ง Podman Desktop (GUI)  
brew install --cask podman-desktop
```

ตรวจสอบการติดตั้ง

```
podman --version
```

การติดตั้ง Podman - Linux (Ubuntu/Debian)

```
# Ubuntu 22.04 ขึ้นไป
sudo apt-get update
sudo apt-get install -y podman

# ติดตั้ง docker compose
pip3 install docker compose
# หรือ
sudo apt-get install docker compose
```

ตรวจสอบการติดตั้ง

```
podman --version
docker compose --version
```

ส่วนที่ 2: คำสั่งพื้นฐานและ Podman Machine

Podman Machine (macOS/Windows)

สร้างและเริ่มต้น Machine

```
# สร้าง machine ใหม่  
podman machine init  
  
# เริ่ม machine  
podman machine start  
  
# ตรวจสอบสถานะ  
podman machine list
```

ตั้งค่า Resources

```
# สร้าง machine พร้อมกำหนด CPU และ Memory  
podman machine init --cpus 4 --memory 4096 --disk-size 50
```

คำสั่งพื้นฐาน - Images

```
# ดึง image จาก registry  
podman pull nginx  
podman pull node:alpine
```

```
# แสดง images ทั้งหมด  
podman images
```

```
# ลบ image  
podman rmi nginx
```

```
# ค้นหา image  
podman search nginx
```

```
# build image จาก Dockerfile  
podman build -t myapp:1.0 .
```

คำสั่งพื้นฐาน - Containers

```
# รัน container
podman run nginx
# รัน container แบบ background
podman run -d nginx

# รัน container พร้อม map port
podman run -d -p 8080:80 --name web nginx
# แสดง containers ที่กำลังรัน
podman ps

# หยุด container
podman stop web
# ลบ container
podman rm web
# ดู logs
podman logs web
podman logs -f web # follow logs
```

คำสั่งพื้นฐาน - Exec และ Debug

```
# เข้าไปใน container  
podman exec -it web /bin/bash
```

```
# ดูสถิติการใช้ทรัพยากร  
podman stats
```

```
# inspect container  
podman inspect web
```

```
# ดู events  
podman events
```

ส่วนที่ 3: Build Image with Dockerfile

Dockerfile คืออะไร?

Dockerfile = ไฟล์สำหรับสร้าง Docker/Podman Image

โครงสร้างพื้นฐาน

```
# Base image (ใช้ latest alpine version)
FROM node:alpine

# Working directory
WORKDIR /app

# Copy files
COPY package.json .

# Run commands
RUN npm install

# Copy application
COPY . .

# Expose port
EXPOSE 3000

# Start command
CMD ["npm", "start"]
```

Dockerfile Instructions

Instruction	คำอธิบาย	ตัวอย่าง
FROM	กำหนด base image	FROM node:alpine
WORKDIR	กำหนด working directory	WORKDIR /app
COPY	Copy ไฟล์เข้า image	COPY . .
RUN	รันคำสั่งขณะ build	RUN pnpm install
EXPOSE	ระบุ port ที่เปิดใช้	EXPOSE 3000
CMD	คำสั่งเริ่มต้น container	CMD ["node", "dist/main"]
ENV	กำหนด environment variable	ENV NODE_ENV=production

Build Image

```
# Build image และตั้งชื่อ
podman build -t myapp:1.0 .

# Build พร้อมระบุ Dockerfile
podman build -t myapp:1.0 -f Dockerfile .

# Build แบบไม่ใช้ cache
podman build --no-cache -t myapp:1.0 .

# ดู build history
podman history myapp:1.0
```


ส่วนที่ 4: Workshop 1 - Hello Node.js

Workshop 1: Hello Node.js

โครงสร้าง Project

```
03-hello-nodejs/  
├── index.js  
├── package.json  
└── Dockerfile
```

เป้าหมาย

1. สร้าง simple Node.js web server
2. เขียน Dockerfile
3. Build image
4. Run container
5. ทดสอบผ่าน browser

index.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/html; charset=utf-8');

  const html = `
    <h1>สวัสดี จาก Node.js!</h1>
    <p>Container กำลังทำงานบน Podman</p>
    <p>Container ID: ${require('os').hostname()}</p>
    <p>Node Version: ${process.version}</p>
  `;

  res.end(html);
});

server.listen(3000, '0.0.0.0', () => {
  console.log('Server running at http://0.0.0.0:3000/');
});
```

package.json

```
{
  "name": "hello-nodejs",
  "version": "1.0.0",
  "description": "Simple Hello World Node.js application",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "keywords": ["nodejs", "hello-world", "podman"],
  "author": "Docker Training",
  "license": "MIT"
}
```

Dockerfile - Hello Node.js

```
# ใช้ Node.js official image (latest alpine)
FROM node:alpine

# กำหนด working directory
WORKDIR /app

# Copy package.json
COPY package.json .

# ติดตั้ง dependencies
RUN npm install --production

# Copy source code
COPY index.js .

# Expose port
EXPOSE 3000

# สั่งรัน application
CMD ["npm", "start"]
```

Build และ Run - Hello Node.js

```
# เข้าไปใน directory
cd workshop/03-hello-nodejs

# Build image
podman build -t hello-nodejs:1.0 .

# ตรวจสอบ image
podman images | grep hello-nodejs

# Run container
podman run -d -p 3000:3000 --name hello-app hello-nodejs:1.0

# ดู logs
podman logs -f hello-app

# ทดสอบ
curl http://localhost:3000
# หรือเปิด browser ไปที่ http://localhost:3000
```

Demo: Hello Node.js

ลองทำตาม

1.  Build image
2.  Run container
3.  เปิด browser ทดสอบ
4.  ดู logs
5.  หยุดและลบ container

```
# หยุดและลบ  
podman stop hello-app  
podman rm hello-app
```

ส่วนที่ 5: Workshop 2 - API with NestJS

Workshop 2: NestJS API

โครงสร้าง Project

```
04-api-nestjs/  
├── src/  
│   ├── main.ts  
│   ├── app.module.ts  
│   ├── app.controller.ts  
│   └── app.service.ts  
├── Dockerfile  
├── package.json  
├── pnpm-lock.yaml  
├── tsconfig.json  
└── nest-cli.json
```

เป้าหมาย

- สร้าง REST API ด้วย NestJS + TypeScript

Dockerfile - Multi-stage Build

```
# Stage 1: Build
FROM node:alpine AS builder
RUN npm install -g pnpm
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile
COPY . .
RUN pnpm run build

# Stage 2: Production
FROM node:alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
EXPOSE 3000
ENV NODE_ENV=production
CMD ["node", "dist/main"]
```

ข้อดีของ Multi-stage Build

Before (Single-stage)

- ขนาด image: ~500 MB
- มี devDependencies ทั้งหมด
- มี source code (.ts files)
- มี build tools

After (Multi-stage)

- ขนาด image: ~150-200 MB
- มีแค่ production dependencies
- มีแค่ compiled code (.js files)
- ไม่มี build tools

API Endpoints

```
// app.controller.ts
@Controller()
export class AppController {
  @Get()
  getRoot() {
    return {
      message: 'สวัสดี จาก NestJS API!',
      endpoints: {
        health: '/health',
        info: '/info',
        users: '/users',
      },
    };
  }

  @Get('health')
  getHealth() {
    return { status: 'OK', timestamp: new Date() };
  }

  @Get('users')
  getUsers() {
    return { users: [...] };
  }
}
```

Build และ Run - NestJS API

```
# เข้าไปใน directory
cd workshop/04-api-nestjs

# Build image (multi-stage build)
podman build -t api-nestjs:1.0 .

# Run container
podman run -d -p 3000:3000 --name api-app api-nestjs:1.0

# ทดสอบ API
curl http://localhost:3000
curl http://localhost:3000/health
curl http://localhost:3000/users
```

Demo: NestJS API

ลองทำตาม

1.  Build image ด้วย multi-stage build
2.  Run container
3.  ทดสอบ endpoints ต่างๆ
4.  เปรียบเทียบขนาด image

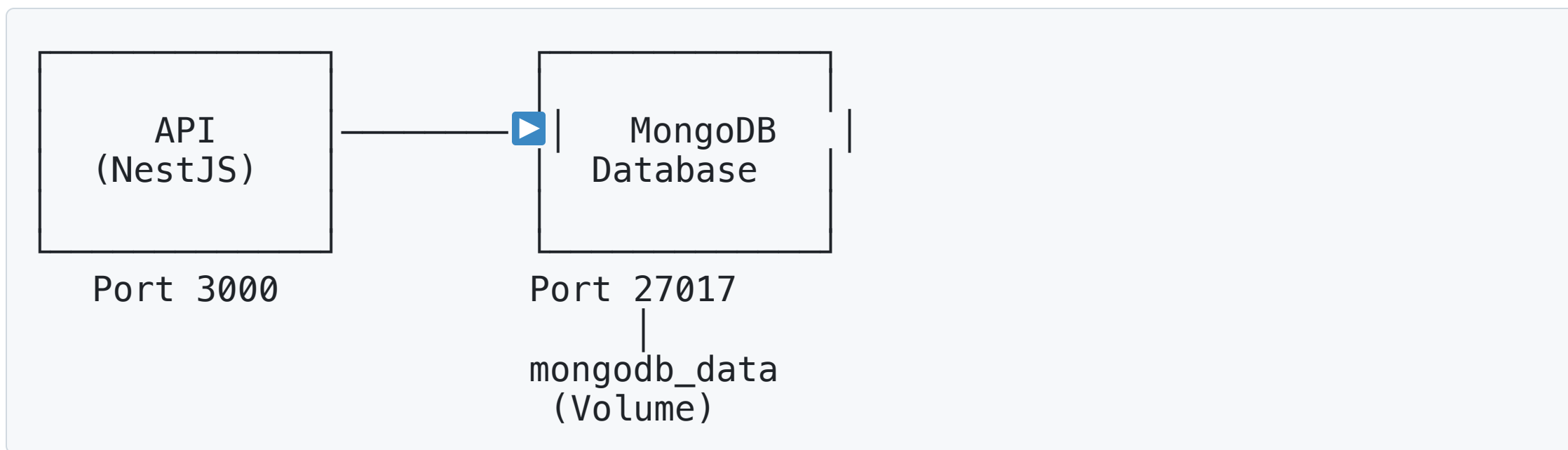
```
# ดูขนาด image
podman images | grep api-nestjs

# เข้าไปใน container ดูโครงสร้าง
podman exec -it api-app /bin/sh
ls -la
```

ส่วนที่ 6: Workshop 3 - Database + Compose

Workshop 3: API + MongoDB

Architecture



เป้าหมาย

- เชื่อมต่อ NestJS API กับ MongoDB
- ใช้ Podman Compose orchestrate services

docker compose.yml

```
version: '3.8'

services:
  mongodb:
    image: mongo:7.0
    container_name: workshop-mongodb
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: password123
      MONGO_INITDB_DATABASE: workshop
    volumes:
      - mongodb_data:/data/db
    networks:
      - app-network

  api:
    build: .
    container_name: workshop-api
    ports:
      - "3000:3000"
    environment:
      MONGODB_URI: mongodb://admin:password123@mongodb:27017/workshop?authSource=admin
    depends_on:
      - mongodb
    networks:
      - app-network

volumes:
  mongodb_data:

networks:
  app-network:
```

Docker Volumes - ทำไมต้องใช้?

ปัญหา: Container ไม่มี Data Persistence

```
# รัน MongoDB
podman run -d --name db mongo:7.0

# เพิ่มข้อมูล
podman exec db mongosh --eval 'db.users.insert({name: "test"})'

# ลบ container
podman rm -f db

# 💥 ข้อมูลหายไปทั้งหมด!
```

วิธีแก้: ใช้ Volume

```
# รัน MongoDB พร้อม volume
podman run -d --name db -v mydata:/data/db mongo:7.0

# ข้อมูลจะถูกเก็บไว้ใน volume 'mydata'
# ลบ container แล้วข้อมูลยังอยู่!
```

คำสั่งจัดการ Volumes

```
# สร้าง volume
podman volume create mydata

# แสดง volumes ทั้งหมด
podman volume ls

# inspect volume
podman volume inspect mydata

# ลบ volume
podman volume rm mydata

# ลบ volumes ที่ไม่ได้ใช้งาน
podman volume prune
```

NestJS API Structure

```
src/
├── main.ts           # Entry point
├── app.module.ts     # Root module
├── app.controller.ts # Root endpoints
├── database/
│   ├── database.module.ts # Database module
│   └── database.service.ts # MongoDB connection
├── users/
│   ├── users.module.ts    # Users module
│   ├── users.controller.ts # CRUD endpoints
│   └── users.service.ts   # Business logic
```

Key Features

- ✓ Modular architecture
- ✓ Dependency injection
- ✓ TypeScript support
- ✓ Auto-initialization with sample data

NestJS Database Service

```
// database.service.ts
@Injectable()
export class DatabaseService implements OnModuleInit {
  private client: MongoClient;
  private db: Db;

  async onModuleInit() {
    const uri = process.env.MONGODB_URI;
    this.client = new MongoClient(uri);
    await this.client.connect();
    this.db = this.client.db('workshop');

    // สร้าง collection และ seed data
    await this.initializeCollections();
  }

  getDb(): Db {
    return this.db;
  }
}
```

Run with Podman Compose

```
# เข้าไปใน directory
cd workshop/05-database-mongodb

# รัน services ทั้งหมด (build + start)
docker compose up -d

# ดู logs
docker compose logs -f

# ดูสถานะ services
docker compose ps

# ทดสอบ API
curl http://localhost:3000/users
curl -X POST http://localhost:3000/users \
  -H "Content-Type: application/json" \
  -d '{"name":"สมชาย","email":"somchai@example.com"}'
```

ตรวจสอบ Volumes

```
# ดู volumes ที่สร้างโดย compose  
podman volume ls
```

```
# ดูรายละเอียด volume  
podman volume inspect 05-database-mongodb-mongodb_data
```

```
# ดูข้อมูลใน MongoDB  
podman exec -it workshop-mongodb mongosh \  
-u admin -p password123 --authenticationDatabase admin
```

```
# ใน mongosh:  
use workshop  
db.users.find()
```

Backup และ Restore Volume

Backup ข้อมูล

```
# Export ข้อมูลจาก MongoDB
podman exec workshop-mongodb mongodump \
  --username admin \
  --password password123 \
  --authenticationDatabase admin \
  --out /data/backup

# Copy backup ออกมา
podman cp workshop-mongodb:/data/backup ./backup
```

Restore ข้อมูล

```
# Copy backup เข้าไปใน container
podman cp ./backup workshop-mongodb:/data/backup

# Restore
podman exec workshop-mongodb mongorestore \
  --username admin \
  --password password123 \
  --authenticationDatabase admin \
  /data/backup
```


Demo: Database + Compose

🎯 ลองทำตาม

1. ✅ รัน services ด้วย docker compose
2. ✅ ทดสอบ API endpoints
3. ✅ เพิ่มข้อมูลผ่าน API
4. ✅ ตรวจสอบข้อมูลใน MongoDB
5. ✅ หยุด containers และรันใหม่ (ข้อมูลยังอยู่)
6. ✅ Backup volume

```
# หยุดและลบ containers (แต่เก็บ volumes)  
docker compose down
```

```
# รันใหม่  
docker compose up -d
```

```
# ข้อมูลยังอยู่!
```

ส่วนที่ 7: Docker Networks

Docker/Podman Network Types

1. Bridge Network (Default)

- Isolated network สำหรับ containers
- Containers สื่อสารกันได้ผ่านชื่อ
- ต้อง publish port (-p) เพื่อเข้าถึงจากภายนอก

2. Host Network

- ใช้ network stack ของ host โดยตรง
- Performance สูงสุด แต่ไม่มี isolation

3. None Network

- ไม่มี network interface
- Maximum isolation

Bridge Network - Default

```
# Containers ที่รันโดยไม่ระบุ network จะใช้ default bridge
podman run -d --name web1 nginx
podman run -d --name web2 nginx

# แต่จะ ping กันไม่ได้!
podman exec web1 ping web2 # ❌ ไม่สำเร็จ
```

ทำไม?

Default bridge ไม่รองรับ automatic DNS resolution

Custom Bridge Network

```
# สร้าง custom network
podman network create my-network

# รัน containers ใน network เดียวกัน
podman run -d --name web1 --network my-network nginx
podman run -d --name web2 --network my-network nginx

# ตอนนี้ ping กันได้แล้ว!
podman exec web1 ping web2 #  สำเร็จ
```

ข้อดี Custom Bridge

-  Automatic DNS resolution (ใช้ชื่อ container ได้)
-  Better isolation
-  ควบคุม network ได้ดีกว่า

Host Network

```
# รัน container ด้วย host network
podman run -d --name web --network host nginx

# nginx จะฟังที่ port 80 บน host โดยตรง
# ไม่ต้องใช้ -p
curl http://localhost:80
```

เมื่อไหร่ควรใช้?

-  ต้องการ performance สูงสุด
-  Application ต้องเข้าถึง network interfaces ของ host
-  ไม่เหมาะสำหรับ multiple containers ที่ใช้ port เดียวกัน

None Network

```
# รัน container โดยไม่มี network
podman run -d --name isolated --network none alpine sleep 3600

# เข้าไปตรวจสอบ
podman exec -it isolated /bin/sh

# ทดสอบ network (จะไม่สามารถเชื่อมต่อได้)
ping 8.8.8.8 # ❌ ไม่สำเร็จ
```

เมื่อไหร่ควรใช้?

-  Batch processing ที่ไม่ต้องการ network
-  Security-sensitive applications
-  Maximum isolation

Network Segmentation

แบ่ง networks ตาม security zones:

```
# สร้าง 3 networks
podman network create frontend-net
podman network create backend-net
podman network create database-net

# Frontend (public-facing)
podman run -d --name web --network frontend-net -p 80:80 nginx

# Backend (internal only)
podman run -d --name api --network backend-net api-app

# Database (most restricted)
podman run -d --name db --network database-net postgres

# เชื่อม api เข้ากับทั้ง frontend และ backend
podman network connect frontend-net api
```


ตรวจสอบ Networks

แสดง networks ทั้งหมด

```
podman network ls
```

ดูรายละเอียด network

```
podman network inspect my-network
```

ดู containers ใน network

```
podman network inspect my-network | grep -A 5 "Containers"
```

ทดสอบ connectivity

```
podman exec web1 ping web2
```

```
podman exec web1 curl http://web2
```

Demo: Networks

ลองทำตาม

1.  สร้าง custom network
2.  รัน containers ใน network
3.  ทดสอบ DNS resolution
4.  เปรียบเทียบ default bridge vs custom bridge
5.  ลอง host network
6.  Network segmentation

Network Best Practices

1. ใช้ Custom Bridge Networks

```
# ❌ ไม่ดี
podman run -d nginx

# ✅ ดี
podman network create my-app-net
podman run -d --network my-app-net nginx
```

2. ตั้งชื่อ Networks ให้สื่อความหมาย

```
podman network create frontend-network
podman network create backend-network
podman network create database-network
```

3. ใช้ Network Segmentation

แบ่ง networks ตาม security zones

สรุป Network Types

Network Type	Isolation	Performance	Use Case
Bridge	✓ ดี	✓ ดี	Default, multi-container apps
Host	✗ ไม่มี	✓✓ ดีที่สุด	High-performance apps
None	✓✓ ดีที่สุด	N/A	Maximum security
Custom	✓✓ ควบคุมได้	✓ ดี	Production apps

สรุปสิ่งที่ได้เรียนรู้

✓ Technical Skills

1. ติดตั้งและใช้งาน Podman
2. เขียน Dockerfile และ build images
3. จัดการ containers, volumes, networks
4. ใช้ Podman Compose orchestrate services
5. เชื่อมต่อ NestJS API กับ MongoDB
6. Backup และ restore data

สรุปสิ่งที่ได้เรียนรู้ (contd.)

✓ Modern Technologies

1. **Latest Node.js** (node:alpine)
2. **pnpm** - Package manager ที่เร็วและประหยัดพื้นที่
3. **NestJS** - Enterprise-grade framework
4. **TypeScript** - Type-safe development
5. **MongoDB** - NoSQL database

✓ Concepts

1. Container vs Image
2. Multi-stage builds
3. Data persistence with volumes

Best Practices สรุป

Production-Ready Containers

1.  ใช้ multi-stage builds
2.  ใช้ .dockerignore
3.  ตั้งค่า health checks
4.  ใช้ volumes สำหรับ data
5.  ใช้ custom networks
6.  ตั้งค่า resource limits
7.  Log management
8.  Security best practices

Next Steps

เรียนรู้ต่อ

1. **Docker Compose** - ไฟล์ compose ที่ซับซ้อนขึ้น
2. **Container Orchestration** - Kubernetes, Docker Swarm
3. **CI/CD** - Build และ deploy containers อัตโนมัติ
4. **Container Security** - Security scanning, best practices
5. **Monitoring** - Prometheus, Grafana
6. **Multi-architecture** - Build สำหรับ ARM, x86

Resources

- Podman Documentation: <https://docs.podman.io>
- Docker Documentation: <https://docs.docker.com>

Q&A และขอบคุณ! 🙏

มีคำถามหรือไม่?

✉️ ติดต่อ

- Email: warut.chm@gmail.com
- GitHub: [WarutC](#)

📁 Workshop Materials

```
# Clone workshop materials  
git clone https://github.com/WarutC/docker-workshop.git
```

ขอบคุณสำหรับการเข้าร่วม Workshop

เจอกันใหม่ในอบรมหน้า 🙌

Happy Containerizing! 🐳🚀